Accreditation Service: Solution Architecture and System Design

Willow Wealth take-home · Supriya Thakur · Design document, version 3 (reviewed; before implementation)

Problem. Admins record and finalise investors' accreditation evidence; the investor app reads the current status. Every change must be auditable through a message broker, and the client view must be built from those events (brief, requirements 1–9).

Shape. One deployable (a modular monolith) on port 9999, built with CQRS and a transactional outbox. Admin commands change the *admin write model* in MySQL; the state change and its event are written in one transaction. A relay publishes events to Kafka, and consumers build the *client read model* that `GET` serves, plus an append-only audit history.

Consistency (Requirement 9b). Rules are checked against the write model, never the eventually consistent projection. A row lock serialises changes to one accreditation, and a pure `decide` function judges every transition. A unique index on a generated column guarantees at most one `PENDING` accreditation per user. All of it holds with any number of instances, because MySQL is the only arbiter.

Delivery. Events are delivered at least once; every consumer is idempotent (version guard, unique `event_id`). A consumer blocks and retries on infrastructure errors and sends only poison events to a dead-letter topic. API retries are safe: an idempotency key stored on the row makes creation exactly-once, and a Redis check rejects a reused key with a different body. Scheduled jobs run on one instance at a time under a MySQL `GET_LOCK`.

Scale (Requirement 9c). The read side already has its own logical database, connection pool and consumers. Client traffic is absorbed in measured steps: index and pool tuning, horizontal copies, read replicas, a versioned cache, and only then a split into separately scaled services.

1. Scope, interpretation and delivery tiers
2. Non-functional requirements
3. Container view, modules and technology
4. Write paths, end to end
5. Accreditation lifecycle
6. Concurrency and invariants (Requirement 9b)
7. API surface and error model
8. Idempotency and delivery semantics

9. Data model and identifiers
0. Event contract and topics
1. Background jobs and threading
2. Security and compliance
3. Failure modes
14. Scaling the client path (Requirement 9c)
15. Test strategy
16. Run and verify
17. Decisions and rejected alternatives
18. Assumptions and scope

1. Scope, interpretation and delivery tiers

#	Brief requirement	Where it is met	How it is verified
API	`POST /user/accreditation`, `PUT /user/accreditation/{id}`, `GET /user/{userId}/accreditation`	Admin and client controllers. Request and response shapes match the brief exactly (snake_case); `GET` returns all of the user's accreditations.	Contract tests that pin the brief's JSON
1	Validate request bodies	Bean Validation, strict enum parsing, base64 and size checks before decoding, rejection of unknown fields; a 400 that lists every invalid field	Parameterised 400 tests
2	Audit history through a messaging system: each event captures the state change and a timestamp	Named events (`AccreditationSubmitted`, `…Confirmed`, `…Failed`, `…Expired`) on `accreditation.events.v1`, kept forever; a consumer appends each to `accreditation_status_history`, exposed by a history endpoint	History endpoint test; duplicate-delivery test
3	A process consumes the events and stores current state in a relational DB, which serves the client endpoint	Projection consumer writes `accreditation_views` in the client read model; `GET` reads only from it	End-to-end test with Awaitility
4a–c	FAILED is final; CONFIRMED can be EXPIRED; no new request while one is PENDING	Pure `decide` under a row lock (4a, 4b); unique index on the generated column `user_id_if_pending` (4c)	Exhaustive transition-table unit tests; concurrency integration tests
5	Expire CONFIRMED accreditations whose last update is older than 30 days	Expiry job, hourly, single runner; each expiry goes through the same command path as `PUT`, under the row lock	Job tests with an injected clock, including a race with an admin update
6–7	Java; containerised broker and DB with their configuration	`docker-compose.yml` (MySQL, Kafka, Redis, topic and seed init, app), multi-stage `Dockerfile`	`run.sh` + `smoke.sh`

8	Public repo, Linux run script, port 9999	`run.sh`, `README.md`, `requests.http`	Smoke script replays the brief's requests
9	README answers 9a–c and an AI statement	This document, condensed into the README	Review

> **Interpretation of requirement 4 and 9b.** The brief says the admin endpoints should "leverage the database containing the entity's current state". Read literally, that is the event-built store from requirement 3. Checks against a store that is updated *after* the request returns cannot be made consistent without making the write wait for the projection. So this design keeps two current-state tables: a **write model**, updated in the same transaction that records the event and used for every check, and the **event-built read model** required by requirement 3, used only by the client endpoint. If the literal reading were mandated, the guarantees in section 6 (the answer to 9b) would still need the synchronous table; the projection could then be fed from it rather than replace it.

Delivery tiers

Tier	Items
Built: core	The three endpoints and validation; write model, row lock, generated-column index; outbox and relay; projection and history consumers with blocking retry and dead-letter topics; expiry job (CONFIRMED > 30 days); MySQL job lock; error model and request ID; idempotency key (Redis check, database unique key); users replica with seed data; unit, integration and contract tests; `run.sh`, `smoke.sh`
Built: extras	JWT security with admin and user tokens (off by default); history endpoint; optional `If-Match` on `PUT`
Designed, not built	Rate limiting (at the gateway); keyset pagination; response cache; read replicas and the service split; tiered retry topics; Schema Registry; CDC relay; object storage and encryption for documents; GDPR crypto-shredding

2. Non-functional requirements

The brief gives no numbers, so these are stated assumptions; they size the design and give section 14 something to measure against.

Assumption	Value
Registered users (investors)	500k, matching Willow's public member count; 5 % active on a peak day
Client `GET`, peak today	50 req/s; 500 req/s after a 10× increase
Admin writes	~2k per day; peak 5 per second
Accreditations per user	1–10; response body < 2 KB
Documents	typically 1 MB, at most 5 MB decoded

Objective	Target
Client `GET` latency	p99 ≤ 150 ms at 10× load
Client `GET` availability	99.9 % per month
Admin write latency (excluding upload transfer)	p99 ≤ 500 ms
Read-model freshness (event time → projection applied)	p99 ≤ 5 s; alert at 30 s
Expiry applied after its threshold	within 1 hour (job cadence)

What the numbers imply. 500 GET/s is 500 single-range index reads per second, well within one MySQL instance; the first limits at 10× are connections and per-request overhead, not the query (section 14). Admin and expiry traffic produce at most a few events per second, so the relay (which drains continuously, section 11) is not a bottleneck.

3. Container view, modules and technology

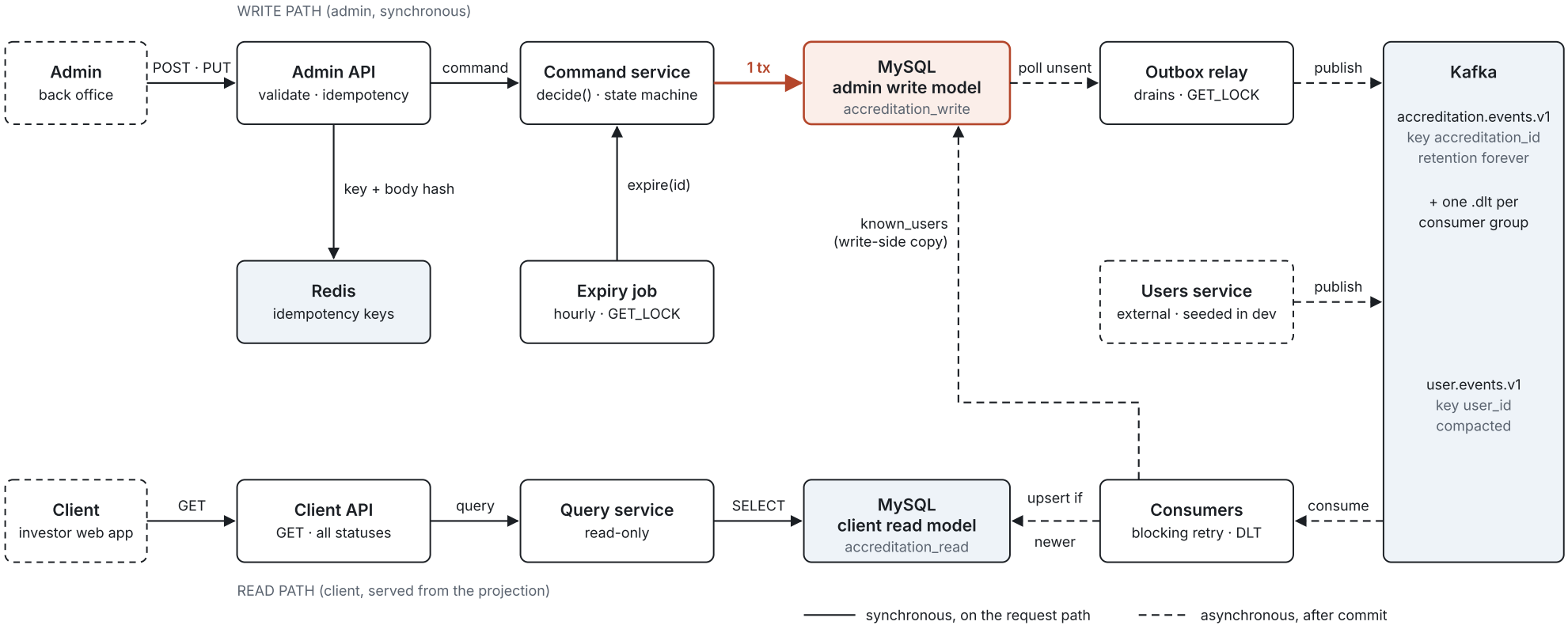


Figure 1. One deployable on port 9999 with two paths. The admin path (top) is synchronous up to one MySQL transaction, highlighted, that changes the state *and* records the event. Everything after the commit (dashed) is asynchronous: the relay publishes to Kafka, and consumers build the client read model that the client path (bottom) serves. Kafka is never on the admin request path, and Redis is never on the client path or under the jobs. Users are owned by another service; the write model keeps a local copy fed by `user.events.v1`, which compose seeds with sample users.

One MySQL server, two logical databases

The service runs against one MySQL server holding two logical databases (schemas): `accreditation_write`, the admin write model, and `accreditation_read`, the client read model. Each has its own datasource, connection pool, database user and Flyway folder. The query module's user has no grant on the write schema, so the boundary is enforced, not conventional; separate pools mean a surge of client reads cannot take the connections admin writes need; and moving the read model to its own server (section 14) changes one JDBC URL. There is no transaction spanning both, by design: the only path between them is the event stream.

Modules inside the deployable

Package	Responsibility	May depend on
`domain`	The functional core: immutable records, sealed types and pure functions. No framework imports, no I/O.	nothing
`command`	Admin API, command service (the shell around `decide`), write repositories, outbox writer and relay, expiry job	domain, platform
`query`	Client API, history API, projection and history consumers, read repositories	domain (event types only), platform
`users`	Consumer of `user.events.v1` that maintains `known_users` in the write model, which POST checks	platform
`platform`	Error model, request-ID filter, idempotency, job lock, security, clock, UUIDv7 generator	nothing in the business modules

`command` and `query` never call each other; Kafka is their only link. An ArchUnit test enforces these rules.

Functional core, imperative shell

```
sealed interface Decision permits Accepted, NoOp, Rejected {}
record Accepted(AccreditationEvent event) implements Decision {}
record NoOp(Accreditation current)       implements Decision {}
record Rejected(RejectionReason reason)  implements Decision {}

// pure: same inputs, same output; no clock, no DB, no exceptions
static Decision decide(Accreditation state, Command cmd, Instant now)
static Accreditation evolve(Accreditation state, AccreditationEvent e)
```

`decide` holds the transition rules for 4a, 4b and the expiry rules; `evolve` applies an accepted event to the write-side state. Business outcomes are values, matched with an exhaustive `switch` over sealed types, not thrown exceptions. The command service is the thin shell: it loads and locks, calls `decide`, persists, and maps the decision to a status code. Rule 4c is deliberately *not* in `decide`: it spans rows, and the database enforces it (section 6).

The projection does not call `evolve`. Each event carries a full snapshot of the accreditation, and the projection applies it if its version is newer, so it tolerates gaps and duplicates. Plain Java 21 only (records, sealed interfaces, pattern matching); no functional library.

Technology choices

Java 21, Micronaut 4.10 (Data JDBC, Security JWT), the plain Kafka client with a small consumer loop (capped in-place retry, explicit offsets), MySQL 8.4, Kafka 3.9 (KRaft), Redis 7, Flyway, Testcontainers.

4. Write paths, end to end

4.1 Submit: `POST /user/accreditation`

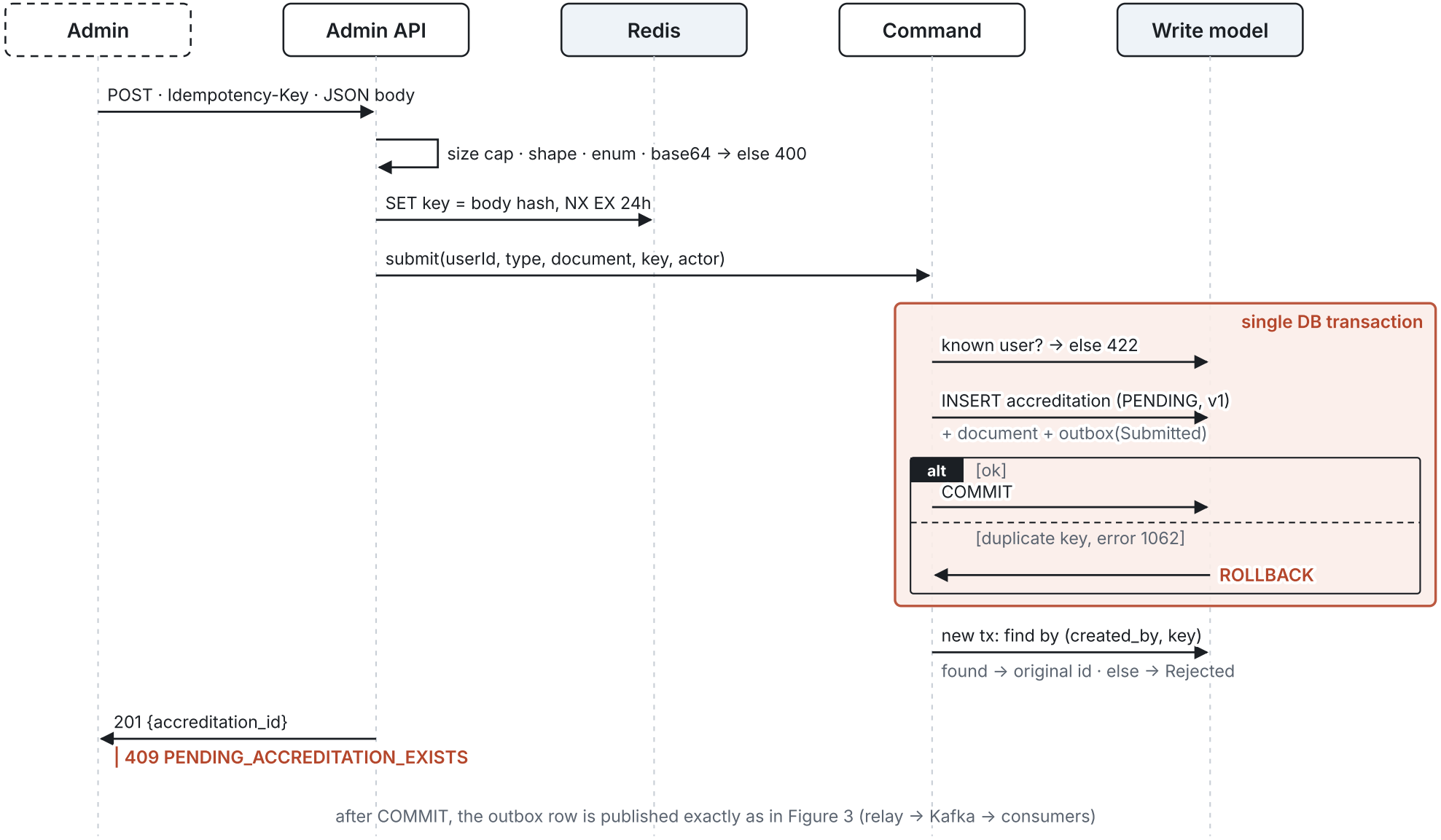


Figure 2. `POST`. The body size and base64 length are checked before decoding, so oversized input fails before any I/O. The known-user check (`SELECT … FOR SHARE`, which holds the user row until commit), the insert and the outbox event share one transaction. Rule 4c is not checked in code first: the unique index on `user_id_if_pending` decides it atomically. Redis rejects

a reused key with a different body (422); the database is the authority for duplicates. A retry can collide on two unique indexes at once (the idempotency key and the pending user), and MySQL reports only one, so every duplicate-key error is resolved the same way: roll back, then look up the idempotency key in a fresh transaction. A match returns the original `accreditation_id` with 201; no match is a genuine second PENDING and returns 409.

4.2 Finalise: `PUT /user/accreditation/{id}`, and what happens after commit

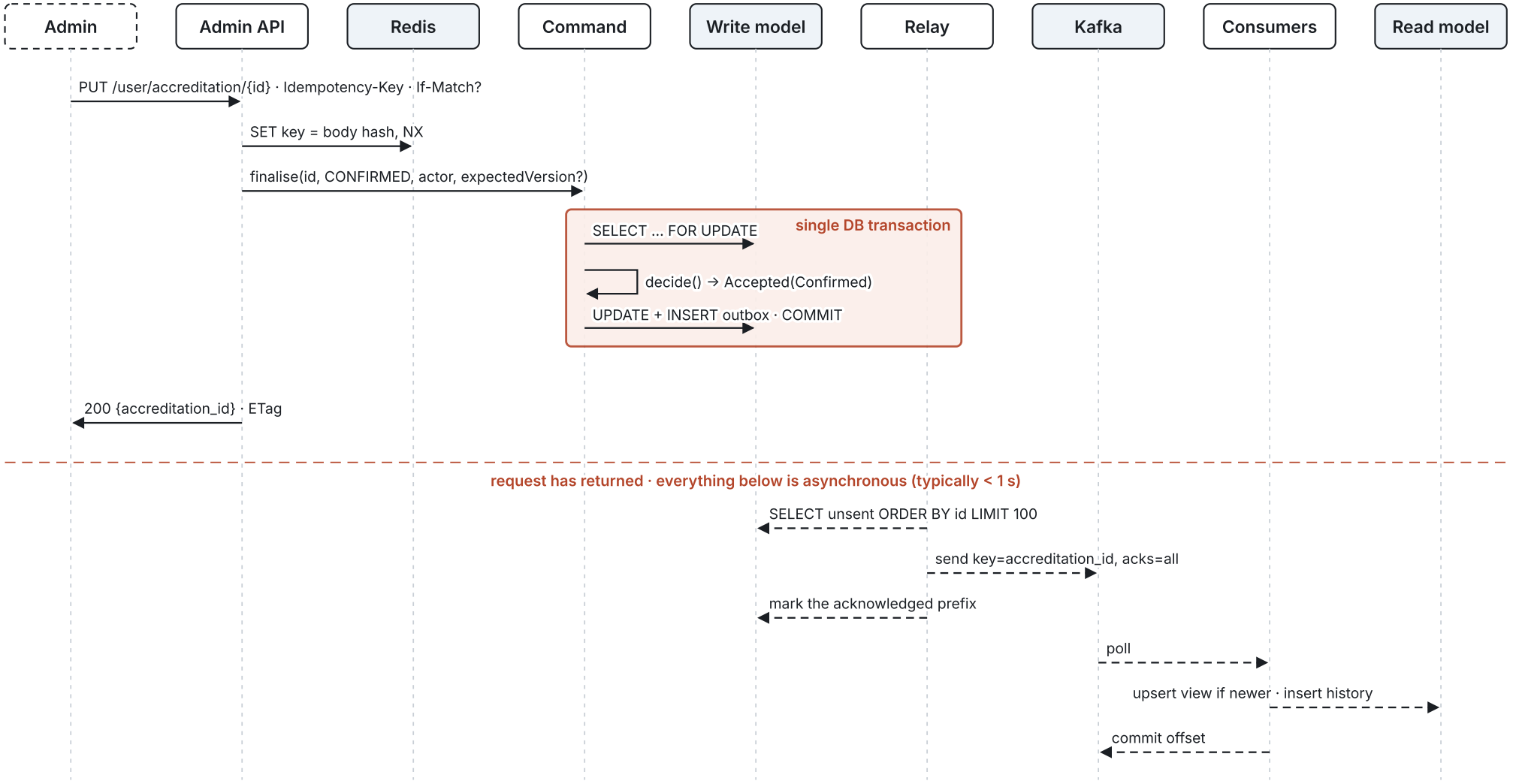


Figure 3. `PUT`, plus the asynchronous half shared by every write. The highlighted transaction is the only place state changes, and it records the event atomically. If the admin sends `If-Match` with the version they reviewed, `decide` also rejects a stale decision (412). The relay stops a batch at the first failed send and marks only the rows Kafka acknowledged, so it never

publishes a later version ahead of an earlier one. Duplicates (a crash between send and mark, or before the offset commit) are absorbed by the version guard and the unique `event_id`.

---

5. Accreditation lifecycle

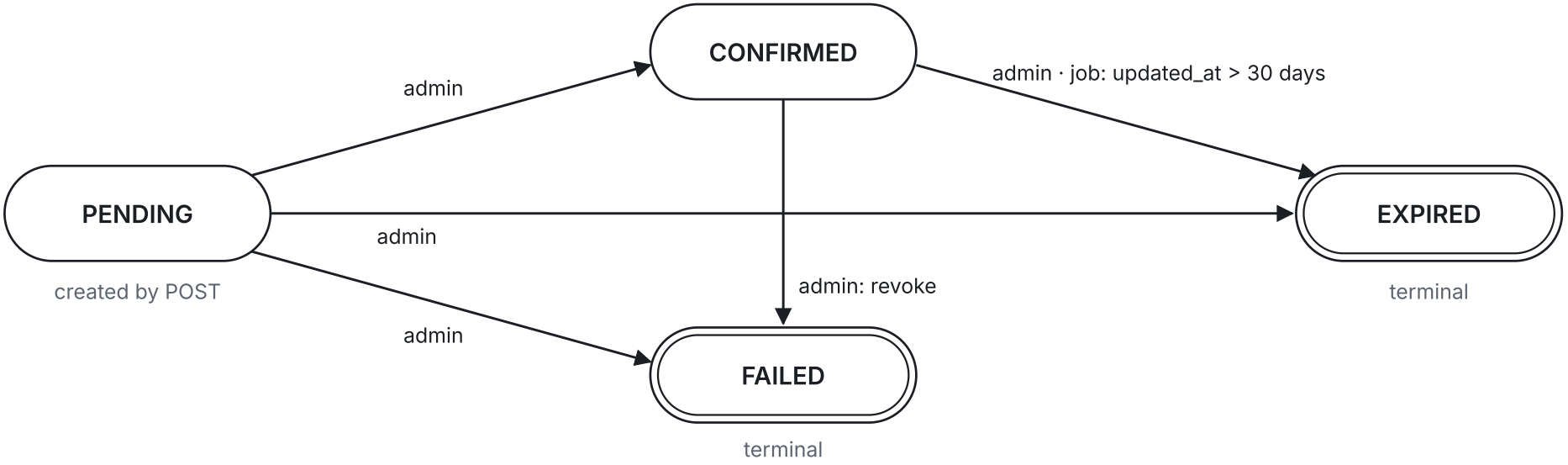


Figure 4. `decide` is the authority on transitions (rule 4c is the database's, section 6). A request that changes a terminal state (FAILED or EXPIRED) returns `409 TRANSITION_NOT_ALLOWED` with the current status in the body, so rule 4a holds. CONFIRMED → FAILED is a revocation by an admin (for example, evidence later found to be forged); it emits `AccreditationFailed` with `previous_status: CONFIRMED`, and the job never takes this edge. Re-sending the current status, in any state, is a `NoOp`: 200, no event, and `updated_at` untouched, so a CONFIRMED accreditation's 30-day clock never restarts and a retried PUT gets the same answer as the original. The expiry job takes only the CONFIRMED → EXPIRED edge.

6. Concurrency and invariants (Requirement 9b)

There are two kinds of invariant, and each gets its own mechanism. Both are enforced by the write-model database inside the command transaction, never by the eventually consistent projection, and both hold for any number of service instances because MySQL is the only arbiter.

Topology: "the only arbiter" means one writable primary per record; app pods scale out, replicas only serve reads, and sharding by `user_id` keeps every user on one primary. **Contention:** row locks make only requests on the same row wait, for a few milliseconds, so parallel activity across users never waits; a per-user Redis lock would add a round trip to every write without removing any database work.

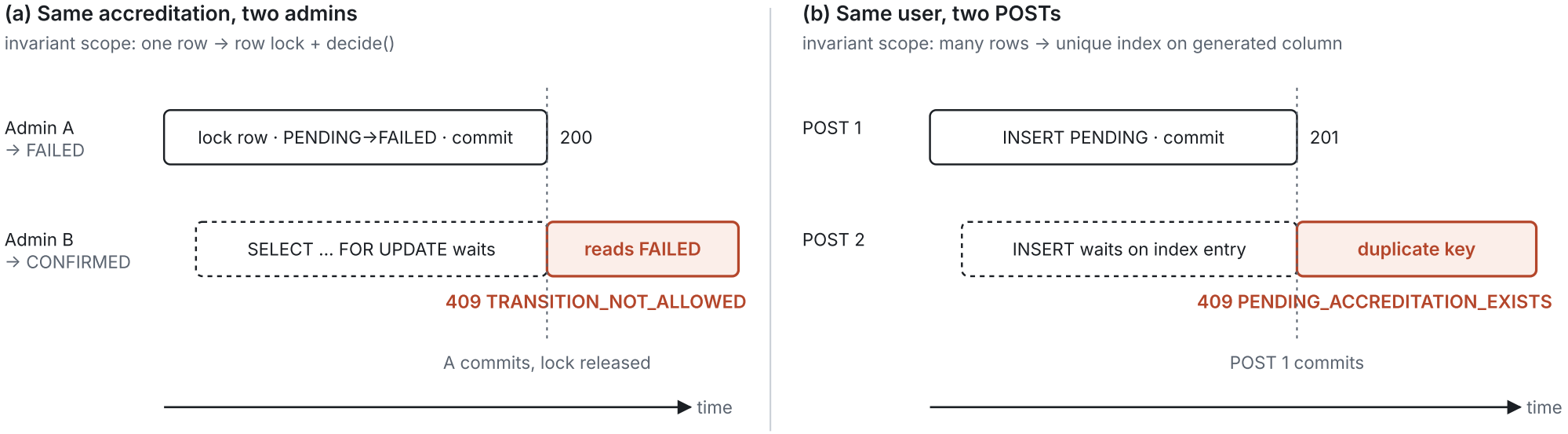


Figure 5. (a) Rules 4a and 4b concern one accreditation, so a pessimistic row lock serialises the writers. `SELECT … FOR UPDATE` is a locking read: it returns the latest *committed* row, not a snapshot, so the loser is judged on the winner's result: FAILED is terminal, so CONFIRMED is rejected with the correct business error. (b) Rule 4c spans rows, so no row lock can cover it. MySQL has no partial unique index, so `user_id_if_pending` is a stored generated column, `CASE WHEN status = 'PENDING' THEN user_id END`, with a unique index; NULLs never collide, so the database allows at most one PENDING per user whichever code path writes.

Interleavings that are handled explicitly

- **Lock, then read.** Every command's first statement on the accreditation is the locking read (`SELECT … FOR UPDATE`); the 404, the `If-Match` check and `decide` all use that result. Anything read earlier, such as the expiry job's scan, is used only to find the row by ID, never to decide.
- **Stale admin decision.** Admin B reviews X as PENDING and decides to fail it; admin A confirms X first; B's FAILED is now legal against CONFIRMED and would silently revoke A's confirmation. With `If-Match: "<version>"` on `PUT` (the version comes from the `ETag` of the last `PUT` or from the history endpoint), B gets 412 instead. The header is optional, so the brief's requests work unchanged.
- **Deadlocks.** Concurrent inserts that collide on a unique index take shared locks, and three of them can deadlock when the first rolls back (MySQL error 1213). The whole transaction is retried up to twice, then answered with 503. A lock-wait timeout (1205, 3 s) rolls back the whole transaction and returns 503.
- **Isolation level.** The write datasource runs at READ COMMITTED. The invariants rest on primary-key locking reads and unique indexes, not on gap locks, and READ COMMITTED avoids gap locks that would otherwise make outbox inserts wait behind cleanup deletes or behind a lookup of a missing ID.

7. API surface and error model

Method and path	Caller	Success	Errors
`POST /user/accreditation`	admin	`201 {"accreditation_id"}`	400 `VALIDATION_FAILED` · 413 body too large · 422 `USER_NOT_FOUND` · 422 `IDEMPOTENCY_KEY_REUSED` · 409 `PENDING_ACCREDITATION_EXISTS` · 503
`PUT /user/accreditation/{id}`	admin	`200 {"accreditation_id"}`, `ETag`	400 `VALIDATION_FAILED` (including outcome `PENDING`) · 404 `ACCREDITATION_NOT_FOUND` (also for a malformed ID) · 409 `TRANSITION_NOT_ALLOWED` · 412 `VERSION_MISMATCH` · 422 `IDEMPOTENCY_KEY_REUSED` · 503
`GET /user/{userId}/accreditation`	client	`200`, the brief's shape with every accreditation; an empty map for a user with none, known or not	400 malformed `userId` · 503
`GET /user/accreditation/{id}/history`	admin	`200`, every transition with its version, in order; the latest version is what an admin sends in `If-Match`	404 · 503
`POST /auth/token`	local stand-in for the admin and user login endpoints	`200 {"access_token"}`, an admin token or a user token	only when security is enabled
`/health`, `/prometheus`, `/swagger-ui`	ops	200	served on the management port, not 9999

All responses carry `X-Request-Id`; a caller-supplied value is accepted only if it matches `[A-Za-z0-9-]{1,64}`, otherwise a new one is generated. It flows to logs, outbox rows and Kafka headers. IDs in paths

are parsed case-insensitively (the brief's example has an upper-case `C`). `401` / `403` appear only when security is enabled. `GET` for an unknown user returns an empty map rather than 404, so the endpoint is not an oracle for which users exist.

Request size. The server's maximum request size is set to 7 MB (5 MB decoded is about 6.7 MB of base64), and the base64 length is checked before decoding. Larger bodies get 413 before they are buffered.

Error body: one envelope for every error

```
// 400
{
  "error_code": "VALIDATION_FAILED",
  "message": "Request body is invalid",
  "request_id": "7f3c9a1e-…",
  "invalid_fields": [
    { "field_name": "accreditation_type",
      "error": "INVALID_VALUE",
      "message": "must be one of BY_INCOME, BY_NET_WORTH" },
    { "field_name": "document.content",
      "error": "INVALID_BASE64",
      "message": "must be valid base64, max 5 MB decoded" }
  ]
}
```

```
// 409
{
  "error_code": "TRANSITION_NOT_ALLOWED",
  "message": "Cannot change status from FAILED to CONFIRMED",
  "request_id": "7f3c9a1e-…",
  "current_status": "FAILED"
}

// 503 adds Retry-After
{ "error_code": "SERVICE_UNAVAILABLE",
  "message": "Please retry", "request_id": "…" }
```

The envelope is snake_case to match the brief's payloads. Unknown fields in requests are rejected; responses may gain fields additively. The paths are fixed by the brief, so the versioning policy is additive change only, with a breaking change going to `/v2`.

8. Idempotency and delivery semantics

Hop	Guarantee	Mechanism
Client → API	A reused key with a different body is rejected	Optional `Idempotency-Key` (1–64 characters). `SET idem:{caller}:{route}:{key} sha256(body) NX EX 24h`, and the result *is used*: if the key already exists with a different hash, 422. Responses are not stored.
POST → DB	Exactly one accreditation per key	`idempotency_key` and `request_sha256` on the accreditation row, unique on `(created_by, idempotency_key)`, written in the insert transaction. A retry, or a concurrent duplicate, returns the original `accreditation_id` with 201 (Figure 2).
PUT retries	The outcome is never applied twice	Re-sending the current status is a `NoOp` (200, no event) in any state, so a retried PUT, including one on FAILED or EXPIRED, gets the same 200 as the original.
Command → outbox	Exactly once, with the state change	Same DB transaction
Outbox → Kafka	At least once; ordered per accreditation	One relay at a time (section 11); ascending outbox `id`; per-aggregate ID order equals commit order because the row lock serialises writes to one accreditation; the relay selects `published_at IS NULL` rather than reading past a high-water mark, so a row that commits late is never skipped. `acks=all`, idempotent producer (which deduplicates retries within one producer session only). A batch stops at the first failed send.
Kafka → projection	Effectively once; never goes backwards	Version-guarded upsert (section 9), then a manual offset commit after the DB commit
Kafka → history	Effectively once; gaps detected	Duplicate `event_id` → ignored. Same `(accreditation_id, version)` with a different `event_id` is a bug → dead-letter topic and alert.

Consumer failures: block and retry, dead-letter only poison

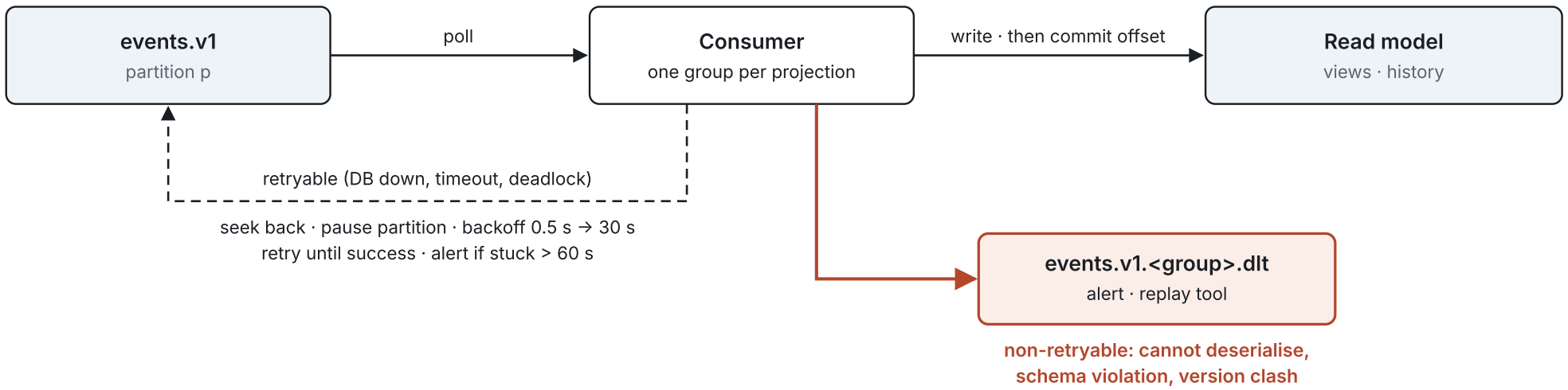


Figure 6. These consumers only write to MySQL, so their failures are either infrastructure-wide (the next record would fail too) or poison. An infrastructure failure is retried *in place*, indefinitely, with exponential backoff: the consumer seeks back to the failed offset and pauses the partition while it waits, so it keeps polling and stays in the group. Per-key order is preserved and a database outage never turns into manual dead-letter work. An unrecognised error gets a bounded number of attempts (5, about 30 s) and then goes to the dead-letter topic, so an unexpected bug can never stall a partition forever. Poison records go straight to the dead-letter topic of that consumer group, with the error in headers; the replay tool re-publishes them to the main topic with the original key. Tiered retry topics (retry after 5 s, then 30 s) are the right tool for consumers that call flaky external services; none do here.

9. Data model and identifiers

Identifiers: UUIDv7, stored as `BINARY(16)`

Accreditation, document and event IDs are UUIDv7, generated by the application: time-ordered, so inserts append to the primary-key index, and not guessable in the API. They are stored as `BINARY(16)` and exposed as the standard 36-character string.

Column conventions

- **`user_id`** is an opaque, case-sensitive token (the brief's `g8NlYJnk…`): `VARCHAR(64) CHARACTER SET ascii COLLATE ascii_bin` in every table, validated at the API as `^[A-Za-z0-9_-]{1,64}$`. MySQL's default collation is case-insensitive and would treat `abcD` and `ABCd` as one user, leaking one user's accreditations to the other and blocking the wrong user under rule 4c.
- **Times** are `DATETIME(6)` in UTC, always set from the application's injected clock (never `ON UPDATE CURRENT_TIMESTAMP`, never mixed with the database clock); the JDBC connection and session time zone are fixed to UTC.
- **Enums** are `VARCHAR` with named `CHECK` constraints rather than MySQL `ENUM`, so adding a value is a plain migration.

Admin write model: `accreditation_write`

Table	Key columns and constraints
`accreditations`	PK `id` · `user_id` · `accreditation_type`, `status` (CHECK) · `version` · `created_by`, `idempotency_key` NULL, `request_sha256`; UNIQUE (`created_by`, `idempotency_key`) · `created_at`, `updated_at` (equal on insert), `updated_by` · `user_id_if_pending` GENERATED STORED, UNIQUE
`accreditation_documents`	PK `accreditation_id` (FK) · `file_name` (sanitised), `mime_type`, `size_bytes`, `sha256` · `content` MEDIUMBLOB
`outbox_events`	PK `id` BIGINT AUTO_INCREMENT (publish order, never exposed) · UNIQUE `event_id` · `aggregate_id` · `event_type` · `payload` (the exact bytes published) · `request_id` · `published_at` NULL
`known_users`	PK `user_id` · `registered_at`

Client read model: `accreditation_read`

Table	Key columns and constraints
`accreditation_views`	PK (`user_id`, `accreditation_id`) · UNIQUE `accreditation_id` · `accreditation_type` · `status` · `version` · `submitted_at` (set on insert only) · `updated_at`
`accreditation_status_history`	PK (`accreditation_id`, `version`) · UNIQUE `event_id` · `event_type` · `from_status`, `to_status` · `performed_by` · `occurred_at`

Indexes and the query each one serves

Index	Serves
`accreditations UNIQUE (user_id_if_pending)`	Rule 4c, enforced by the database; NULLs (rows that are not PENDING) never collide
`accreditations UNIQUE (created_by, idempotency_key)`	Duplicate POST detection, scoped per caller like the Redis key; NULL keys never collide
`accreditations (status, updated_at)`	Expiry job: keyset walk `WHERE status = ? AND updated_at < :cutoff AND (updated_at, id) > (:last_at, :last_id) ORDER BY updated_at, id LIMIT 500`
`outbox_events (published_at, id)`	Relay: `WHERE published_at IS NULL ORDER BY id LIMIT 100` without a filesort; cleanup in primary-key batches of 500, with `published_at < cutoff` repeated in the `DELETE`
`accreditation_views` clustered PK `(user_id, accreditation_id)`	Client `GET`: a user's rows are physically adjacent, so the read is one short range scan with no secondary lookups
`accreditation_status_history` PK `(accreditation_id, version)`	History endpoint in version order; a second event for the same version is rejected

Applying an event to the projection

```
-- 1. apply the event only if it is newer than what we hold
UPDATE accreditation_views
   SET status = ?, version = ?, updated_at = ?
 WHERE accreditation_id = ? AND version < ?;

-- 2. if no row was updated, insert it
INSERT INTO accreditation_views
  (user_id, accreditation_id, accreditation_type,
   status, version, submitted_at, updated_at)
VALUES (?, ?, ?, ?, ?, ?, ?);
-- duplicate key here: the row exists with the same or a
-- newer version, so the event is ignored
```

A guarded `UPDATE` followed, only if it matched nothing, by a plain `INSERT`. A duplicate event, or one older than the version held, changes nothing; the projection never goes backwards. Each partition has one consumer thread, so two events for the same accreditation are never applied at once; the caught duplicate-key error covers the case anyway.

History inserts catch the duplicate-key error and check which key fired, rather than using `INSERT IGNORE`, which would also hide other errors.

The document sits in its own table, so status reads never touch the blob. It is decoded and hashed before the transaction begins, so the blob write does not lengthen the time the unique-index lock is held. In production it would go to object storage (section 12), with only the checksum and key kept in the DB. Events never carry the file.

10. Event contract and topics

```
// topic accreditation.events.v1, key = accreditation_id
// headers: X-Request-Id, event_type
{
  "event_id": "0192a1d0-6f1d-7c3a-9a51-3c1f0b7d2e10",
  "event_type": "AccreditationConfirmed",
  "schema_version": 1,
  "accreditation_id": "0192a1c9-5a20-7b11-8f0e-6a2d9c4b7f31",
  "user_id": "g8NlYJnk7zK9BlB1J2Ebjs0AkhCTpE1V",
  "accreditation_type": "BY_INCOME",
  "previous_status": "PENDING",
  "status": "CONFIRMED",
  "version": 2,
  "submitted_at": "2026-10-12T08:01:02.000000Z",
  "occurred_at": "2026-10-15T09:12:44.123456Z",
  "performed_by": "admin-7"
}
```

Named facts. A sealed family: `AccreditationSubmitted` (version 1, carries `document_sha256`, no previous status), `AccreditationConfirmed`, `AccreditationFailed` and `AccreditationExpired`. Each reads as an entry in an audit trail, and consumers switch on them exhaustively. The expiry job's events carry `performed_by: "system:expiry-job"`.

Snapshot fields. Every event carries the full current state (user, type, status, version, `submitted_at`), so a consumer can build or repair its row from any single event. `previous_status` makes each event a complete from→to change, as requirement 2 asks, without a lookup.

Evolution. Additive changes only within `v1`; consumers ignore unknown fields. A breaking change means a new topic (`v2`), written in parallel during migration. Production would enforce this with Avro or Protobuf and a Schema Registry in backward-compatibility mode; here events are versioned JSON, readable with `kafka-console-consumer`.

Audit record. The topic is the system of record for the audit trail: it is append-only and kept forever. The history table is a queryable projection of it and can be rebuilt from it.

Topics

Topic	Key	Partitions	Cleanup / retention	Replication
`accreditation.events.v1`	`accreditation_id`	6	delete, retention forever	Compose: 1 broker, RF 1. Production: RF 3, `min.insync.replicas` 2, unclean leader election off
`accreditation.events.v1.<group>.dlt`	same	1	delete, 30 days	
`user.events.v1`	`user_id`	3	compact (a deleted user is a tombstone)	
`user.events.v1.<group>.dlt`	same	1	delete, 30 days	

Partition counts. Throughput needs one partition (a few events per second at peak). The counts are sized for consumer parallelism, since a group has at most one active consumer per partition, and so they never need to grow: adding partitions to a keyed topic moves keys and breaks per-key ordering during the change. Two per broker on a three-broker production cluster gives 6; user events are rarer, so 3.

Automatic topic creation is off; a compose init container creates the topics and publishes `UserRegistered` for the brief's sample user and a few others. Consumer groups: `accreditation-projection`, `accreditation-history`, `known-users`. The users topic is compacted, so only the latest record per user is guaranteed: the `known-users` consumer treats any record as "this user exists" and a tombstone as a deletion, never depending on a particular earlier event type.

11. Background jobs and threading

Job	Schedule	What it does
Outbox relay	every 500 ms	Drains: repeats batches of 100 while batches are full; sends a batch asynchronously, flushes, then marks the acknowledged rows in order up to the first failure. Producer timeouts (`delivery.timeout.ms` 30 s) bound a slow broker. Deletes published rows older than 7 days in primary-key batches of 500.
Expiry	hourly (configurable)	Computes the cutoff once per run from the injected clock, then walks `CONFIRMED` rows with `updated_at` older than 30 days, using the keyset walk in section 9. The scan collects IDs only. Each ID gets its own short transaction: row lock, then eligibility re-checked on the locked row (status is CONFIRMED *and* `updated_at` is before the cutoff), then `decide`, update and outbox event. A row an admin changed after the scan fails the re-check and is skipped, so nothing is ever decided on the scanned values.

One runner per job. Every job method carries `@ScheduledJobLock`. An interceptor runs `SELECT GET_LOCK('job:Class.method', 0)` on a dedicated connection: the first pod gets the lock and runs; every other pod gets 0 and *skips* that run instead of queuing. The lock is held by the connection, so it is released when the job finishes, when the pod dies, or when the connection drops; there is no lease that can expire mid-run. Jobs need MySQL anyway, so this adds no dependency and Redis is never under a job. A MySQL failover drops the lock; a second relay could then overlap briefly, which the consumers absorb (duplicates and reordering are handled by the version guard). If one runner ever becomes too slow, `SELECT … FOR UPDATE SKIP LOCKED` lets several pods share the work.

Threading on Java 21

Work	Runs on	Why
HTTP handlers	Virtual threads (`@ExecuteOn(TaskExecutors.BLOCKING)`, backed by virtual threads on Java 21)	Handlers block on JDBC and Redis; blocking a virtual thread is cheap, so concurrency is bounded by the connection pools, which are the real limit. A per-route semaphore caps in-flight requests so overload is shed with 503 rather than queued.
Expiry job, per accreditation	A virtual-thread-per-task executor with a `Semaphore` of half the write pool	Each expiry is an independent short transaction; the semaphore keeps the job from starving admin writes of connections.
Kafka poll loops	One platform thread per consumer	`KafkaConsumer` is not thread-safe; parallelism comes from partitions.
Outbox relay	The scheduler thread	Deliberately sequential, so publish order is preserved.

Virtual threads raise how many requests can wait on I/O at once; they do not make a single request faster. Code on virtual threads avoids `synchronized` around blocking calls, which would pin the carrier thread on Java 21.

---

12. Security and compliance

Accreditation evidence (tax returns, bank and brokerage statements) is sensitive personal and financial data. Security is fully built but **off by default**, so the brief's requests run unchanged; `SECURITY_ENABLED=true ./run.sh` turns it on, and the integration tests run with it on.

Concern	Design
Authentication	JWT bearer tokens, signed and checked for expiry. Two kinds: an **admin token** and a **user token** (carrying the `user_id`). In production, admins and users obtain them by logging in with email and password at the authentication service, which is out of scope here; locally, `POST /auth/token` stands in for those logins.
Authorisation	Admin endpoints and history require an admin token. `GET /user/{userId}/accreditation` accepts an admin token, or a user token for that same `userId`, so a user can never read another user's accreditations.
Audit actor	`performed_by` and `created_by` come only from the token subject, never from the body or a header; with security off they are `anonymous`. The idempotency scope uses the same identity.
Operational endpoints	Health, metrics and Swagger UI on a separate management port, not on 9999.
Documents	Size-capped before decoding; MIME type from an allow-list (PDF, PNG, JPEG); `file_name` sanitised (length, path separators, control characters). Documents are never served back by this service. Production: object storage with server-side envelope encryption (KMS) and malware scanning on upload; the database keeps only the key and checksum.
Logging	Request bodies and `document.content` are never logged; the HTTP log redacts them. Logs carry IDs and the request ID only.
Retention and erasure	Accreditation records fall under regulatory record-keeping obligations that can conflict with GDPR erasure requests. Events carry only `user_id` and the document checksum, no personal details. Production would encrypt any personal fields per user and destroy the key on erasure (crypto-shredding), so the permanent topic stays append-only; the users topic gets a tombstone.
Infrastructure	Compose uses development credentials. Production: secrets from a vault or Kubernetes secrets; TLS everywhere; Kafka and Redis require each service to authenticate, and grant it only the topics, consumer groups and keys it needs; least-privilege database users per schema (the query module has no grant on the write schema).
Input hygiene	Strict JSON (unknown fields rejected), validated IDs, a validated or regenerated `X-Request-Id` so callers cannot inject into logs or headers.

13. Failure modes

Failure	Effect	Behaviour and alert
Kafka unavailable	Admin API unaffected; client view and audit history go stale	Outbox rows accumulate (about 1 KB each; a day of peak writes is a few MB) and the relay drains them on recovery. Alert on outbox age > 30 s.
MySQL unavailable	Reads and writes fail; consumers pause	`503` with `Retry-After`; pool acquire timeouts are short so requests are not held. Consumers retry in place (Figure 6) and resume where they stopped.
Redis unavailable	No check for a reused key with a different body	Requests run on the database guarantees (POST deduplicated by its unique key; PUT retries are a `NoOp` returning 200). Short Redis timeouts (100 ms, no retries) keep latency flat. Jobs and the client path do not use Redis.
Lock wait (3 s) or deadlock	One request fails	Deadlock: whole transaction retried up to twice. Otherwise `503` with `Retry-After`; the retry is safe.
Relay crash after send, before mark	Duplicate publish	Consumers are idempotent
Consumer crash before offset commit	Redelivery	Version-guarded upsert; unique `event_id`
Poison event	That event is set aside; the partition continues	Dead-letter topic of the group; alert when any dead-letter topic is non-empty
Consumer lag	Client view stale	Alert when freshness (event time → applied) exceeds 30 s
Two pods fire the same job	None	`GET_LOCK` with zero wait; the second skips
Expiry job down for a day	Accreditations stay CONFIRMED past 30 days	Alert on "last successful expiry run" > 2 h. Eligibility checks at investment time should also treat a CONFIRMED older than 30 days as expired.

Projection needs rebuilding	None for clients	Build a new table from the earliest offset with a new consumer group, then switch reads to it (blue/green); the topic keeps events forever

14. Scaling the client path (Requirement 9c)

Client traffic produces no events, so the client path scales independently of Kafka and of the admin path. Each step below is taken only when its trigger shows up in the metrics, cheapest first, against the objectives in section 2.

Step	Change	Trigger
1. Measure	Load test at 1×, 5× and 10× of the section 2 baseline; dashboards for per-route rate, errors and duration, connection-pool wait, consumer lag and freshness	Always first
2. Make each GET cheap	Keep each GET to one indexed read (section 9), size the read connection pool, and answer 503 quickly when it is full instead of letting requests queue	Slow queries or requests waiting for a connection
3. Run more copies	The service is already stateless (jobs are single-run under `GET_LOCK`), so add replicas behind the load balancer. Client routes get their own bulkhead (thread and connection limits); note that copies still share each instance's CPU and heap between admin and client work	CPU > 70 % or in-flight requests at the cap
4. Read replicas	Point the read datasource at replicas of the read-model database. Replica lag adds to freshness, and two replicas can be at different points, so a user could see CONFIRMED then PENDING; route each user to one replica (consistent hashing) to keep reads monotonic	Read-DB CPU > 60 % or its share of p99
5. Cache, if it pays	A *separate* Redis (`allkeys-lru`, unlike the idempotency Redis's `noeviction`). Each entry carries the user's highest version, and both the projector and a reader that misses write it only if newer (a small script), so a stale fill can never overwrite a newer value; TTL 60 s; one in-flight load per key	Measured hit ratio above ~70 % and replicas becoming the cost driver
6. Split the deployable	Command service (small), query service scaled on CPU and in-flight requests with a non-zero minimum and a readiness delay for JVM warm-up, and the projector as its own deployment sized to the partition count (so client scaling never rebalances consumers). Total connections, instances × pool, stay under the database's `max_connections`, or a proxy pools them	GC or CPU interference between paths, or different deploy cadences
7. Edge	Gateway: authentication once, rate limits per user or IP (429)	Abusive or bursty clients

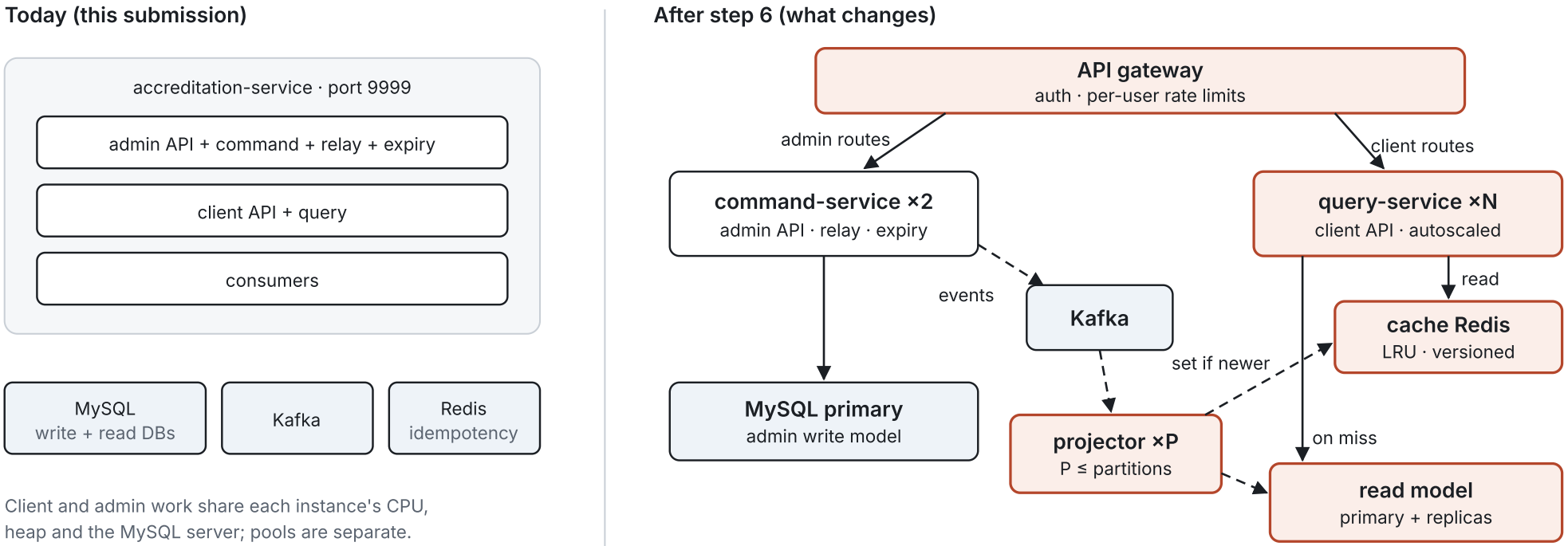


Figure 7. Highlighted elements are what the scaled design adds; they arrive one step at a time and only on their triggers. The read model already has its own logical database, datasource and consumers, so separating it is configuration. Partitions are not a 9c lever: client traffic adds no events, and the topic is created with 6 partitions so projector parallelism never requires repartitioning.

15. Test strategy

Level	What it proves	Examples
Unit (no containers)	Business rules and pure code	Every (state × command) pair through `decide`, including terminal states and a repeat of the current status in every state (200 no-op), expiry of an accreditation that is no longer CONFIRMED or was updated after the cutoff, and `If-Match`; UUIDv7 format and ordering; validation and error mapping
Contract	The brief's API, byte for byte	The brief's exact POST, PUT and GET payloads; response field names and shapes; status codes for each error
Integration (Testcontainers: MySQL, Kafka, Redis)	The wiring and the real database behaviour	End to end POST → PUT → GET and history; confirm then fail (revocation): `AccreditationFailed` with `previous_status: CONFIRMED` reaches GET and history; outbox published after a Kafka outage; projection with duplicate and out-of-order events; poison event to the dead-letter topic; consumer blocks while MySQL is stopped and resumes; collation (`abcD` ≠ `ABCd`); expiry job: `AccreditationExpired` is published, GET shows EXPIRED, and history records it with `system:expiry-job`
Concurrency	The 9b guarantees under real contention	20 parallel PUTs with mixed outcomes on one PENDING: every accepted change is a legal transition from the state committed just before it, with exactly one event each, and once FAILED or EXPIRED every later request for a different status gets 409; parallel POSTs for one user: exactly one PENDING; expiry job racing an admin revocation; deadlock retry; two job instances: one run
Idempotency	Retries are safe	A retried POST returns the original ID with 201; concurrent duplicates create one accreditation; same key and different body: 422; key reuse across admins does not collide
Security (enabled)	Access rules	401 without a token; 403 for a user reading another user; admin-only routes; `performed_by` from the token
Architecture	Module boundaries	ArchUnit: `command` and `query` never depend on each other; `domain` has no framework imports
Smoke	The packaged stack	`smoke.sh` replays the brief's requests against `run.sh` and polls for the eventually consistent GET

`./mvnw verify` runs everything except the smoke test and needs only Docker; CI runs the same command. `run.sh` does not run the tests, so starting the service stays fast.

---

16. Run and verify

- **Prerequisites:** Docker with Compose v2. No JDK: the image is built in a multi-stage Dockerfile.
- **`./run.sh`** builds the image and starts the stack in order, each step waiting for the previous one's health check: MySQL → Kafka → init (creates the topics with the partition counts in section 10, seeds sample users) → service (Flyway migrations on start). It prints the URLs once the service is healthy on `http://localhost:9999`.
- **`./smoke.sh`** runs the brief's requests and prints each response.
- **`requests.http`** has every request, including the security-enabled flow with admin and user tokens.
- **Watching events:** `docker compose exec kafka kafka-console-consumer.sh --bootstrap-server localhost:9092 --topic accreditation.events.v1 --from-beginning`
- **`./run.sh down`** stops the stack; `./run.sh clean` also removes its volumes.

```
# the brief's flow, end to end
curl -s -X POST localhost:9999/user/accreditation \
  -H 'Content-Type: application/json' -d '{
  "user_id": "g8NlYJnk7zK9BlB1J2Ebjs0AkhCTpE1V",
  "accreditation_type": "BY_INCOME",
  "document": { "name": "2018.pdf",
    "mime_type": "application/pdf",
    "content": "ICAiQC8qIjogWyJzcmMvKiJdCiAgICB9CiAgfQp9Cg==" }}'
# → 201 {"accreditation_id":"0192…"}

curl -s -X PUT localhost:9999/user/accreditation/0192… \
  -H 'Content-Type: application/json' -d '{"outcome":"CONFIRMED"}'

curl -s localhost:9999/user/g8NlYJnk7zK9BlB1J2Ebjs0AkhCTpE1V/accreditation
# eventually consistent: normally visible within a second
```

17. Decisions and rejected alternatives

Decision	Why	Rejected	Cost accepted
CQRS with a transactional outbox	Checks stay strongly consistent; the brief's event-built read model is met; nothing is ever written to only one of DB and Kafka	Publish first and let a consumer build state (stale checks); dual write; full event sourcing (the cross-row rule 4c needs a uniqueness reservation, plus upcasting and replay tooling)	An extra table and a relay; client reads are eventually consistent
Row lock and a pure `decide`	The invariant is enforced where it lives; deterministic business errors; short transactions	Redisson lock per accreditation; optimistic version with retry; single writer per key via Kafka	A waiting writer holds a DB connection, bounded by a 3 s lock wait
Unique index on a generated column for 4c	Atomic across rows, whichever code path writes	Check-then-insert (races); a per-user distributed lock	MySQL-specific DDL
READ COMMITTED on the write datasource	No gap locks between cleanup, lookups and inserts; the invariants do not rely on them	The default REPEATABLE READ	Two reads in one transaction can see different data; no rule here relies on them matching
UUIDv7 as `BINARY(16)`	No coordination, time-ordered inserts, opaque in the API, known before insert	AUTO_INCREMENT (guessable, leaks volume); 64-bit time + node + sequence (needs node-ID coordination); UUIDv4 (random inserts split index pages)	16-byte keys; `BIN_TO_UUID` when reading SQL by hand
Idempotency key: Redis check, database unique key	Exactly-once creation guaranteed by the DB; Redis rejects a reused key with a different body; no stored responses	Storing and replaying responses (claim states, tokens, more failure cases); Redis only (not the authority)	Response bodies are rebuilt rather than replayed (they contain only the ID)
Blocking retry, then a dead-letter topic	Outages heal on their own without manual replay; per-key order	Tiered retry topics (break ordering, turn an outage into dead-letter work, and are hand-built in this	A stuck record delays its partition, surfaced by an

	preserved	framework)	alert
Job lock with MySQL `GET_LOCK`	Released with the connection; no lease; no extra dependency under the jobs	Redisson lock (Redis outage halts the relay and expiry; leases can expire mid-run); ShedLock; Quartz clustering	The lock does not survive a MySQL failover (overlap is absorbed)
Kafka key `accreditation_id`, 6 partitions up front	Per-aggregate ordering, even spread, no repartitioning later	`user_id` key (hot keys; no rule needs per-user ordering)	No ordering across one user's accreditations
Modular monolith; one MySQL server, two logical DBs	One thing to run and review; enforced boundaries; splitting is configuration	Microservices now; one schema	Shared CPU and heap until the split
Users: local copy in the write model from `user.events.v1`, seeded in dev	No synchronous dependency on the request path; unknown users rejected on POST (422)	REST call to the Users service; no user check at all	A newly registered user appears after a short delay
Named-fact events with snapshot fields, versioned JSON	Reads like an audit trail; consumers tolerate gaps; readable on the wire	One generic "status changed" event; Avro or Protobuf with a Schema Registry (production choice)	Compatibility enforced by convention and tests
Functional core in plain Java 21	Rules are pure and exhaustively testable; outcomes are values	Vavr; exceptions for business outcomes	Some ceremony when collecting validation errors
Micronaut Data JDBC with Flyway	Compile-time repositories; explicit SQL where it matters; maps to Spring Data JDBC	jOOQ with code generation (needs a database at build time); JPA	The important SQL lives in `@Query` strings, covered by integration tests
JWT built, off by default	The brief's requests run unchanged; security is one flag away and tested	Always on; none	A flag to keep honest
Pagination and rate limiting designed, not built	The brief asks for all statuses (a user has a handful); keyset pagination, not offset, if that	Building both now	Described, not demonstrable

	changes. Rate limits belong at the gateway		
Blocking I/O on virtual threads	Simple, debuggable code at high concurrency; JDBC blocks anyway	Reactive (R2DBC, Reactor)	Avoid `synchronized` around blocking calls

18. Assumptions and scope

Assumptions

- Allowed transitions: PENDING → CONFIRMED, FAILED or EXPIRED; CONFIRMED → EXPIRED or FAILED (revocation). The brief forbids only updates to FAILED; it does not forbid CONFIRMED → FAILED.
- PENDING → EXPIRED is allowed ("or the accreditation status has expired").
- EXPIRED is terminal: once evidence has expired, the user submits fresh evidence as a new accreditation with its own audit trail, rather than an old one being revived.
- FAILED and EXPIRED cannot be changed (409). Re-sending the current status, in any state, is a no-op returning 200; it emits no event and does not restart the 30-day clock.
- The 30-day clock runs from `updated_at`, which equals `created_at` on insert.
- A user with a CONFIRMED or EXPIRED accreditation may submit another; only PENDING blocks.

Designed, not built (README next steps)

- Gateway rate limiting; keyset pagination; response cache
- Automatic expiry of PENDING accreditations left unreviewed, so they cannot accumulate and block resubmission
- Processed-request records written in the same transaction as each change, so any retried request returns exactly its original result
- Read replicas, the service split and autoscaling (section 14)
- Object storage with encryption and malware scanning; crypto-shredding
- Schema Registry; CDC (Debezium) instead of the polling relay
- OpenTelemetry tracing across Kafka (the request ID is propagated today)
- Admin listing and search endpoints

- `user_id` is an opaque, case-sensitive token issued by the Users service.
- POST for an unknown user returns 422; GET for a user with no accreditations returns an empty map.
- Documents: PDF, PNG or JPEG by declared MIME type, at most 5 MB decoded. The brief's sample content is placeholder text (it decodes to a `tsconfig.json` fragment), so bytes are not sniffed.
- Requests with unknown JSON fields are rejected.
- The client `GET` is eventually consistent, normally within a second of an admin write.